Bash Scripting Fundamentals for DevOps

Introduction

Bash (Bourne Again SHell) is a command-line interpreter that allows users to interact with their operating system. Bash scripting enables automation of repetitive tasks, system administration, and deployment processes, making it an essential skill for DevOps professionals.

Why Bash is Important in DevOps

- **Automation**: Eliminates manual, repetitive tasks
- **Consistency**: Ensures processes run the same way every time
- **Efficiency**: Saves time and reduces human error
- **Integration**: Works seamlessly with other DevOps tools
- **Portability**: Available on most Unix-like operating systems

Basic Syntax

Creating Your First Script

```
#!/bin/bash              # this is 'Shebang'
# This is a comment
echo "Hello, World!"
```

Save this as `hello.sh` and make it executable:

chmod +x hello.sh
./hello.sh

Variables

Defining and Using Variables

```
#!/bin/bash

# Assign values to variables
name="DevOps Engineer"
number=42
```

```
# Access variables using $
echo "I am a $name"
echo "My favorite number is $number"

# Command substitution
current_date=$(date)
echo "Today is $current_date"

# Arithmetic operations
a=5
b=3
sum=$((a + b))
echo "Sum: $sum"
```

Special Variables

```
#!/bin/bash

echo "Script name: $0"
echo "First argument: $1"
echo "Second argument: $2"
echo "All arguments: $@"
echo "Number of arguments: $#"
echo "Process ID: $$"
echo "Exit status of last command: $?"

Run this script with: ./script.sh arg1 arg2
```

Conditionals

If-Else Statements

```
#!/bin/bash

# Check if a file exists
if [ -f "config.txt" ]; then
    echo "Configuration file exists"
```

```
else
    echo "Configuration file not found"
    touch config.txt
    echo "Created new configuration file"
fi

# Numeric comparison
age=25
if [ $age -gt 18 ]; then
    echo "Adult"
else
    echo "Minor"
fi

# String comparison
username="admin"
if [ "$username" = "admin" ]; then
    echo "Administrator privileges granted"
fi
```

Case Statements

```
#!/bin/bash

echo "Select an option (1-3):"
read option

case $option in
    1)
        echo "You selected option 1"
        ;;
    2)
        echo "You selected option 2"
        ;;
    3)
        echo "You selected option 3"
        ;;
    *)
        echo "Invalid option"
        ;;
esac
```

Loops

For Loops

```
#!/bin/bash

# Loop through a range of numbers
echo "Counting from 1 to 5:"
for i in {1..5}; do
    echo "Number: $i"
done

# Loop through items in a list
echo "Looping through services:"
for service in nginx mysql docker; do
    echo "Checking $service service..."
done

# Loop through files
echo "Log files in directory:"
for file in /var/log/*.log; do
    echo "Log file: $file"
done
```

While Loops

```
#!/bin/bash

# Count down from 5
count=5
while [ $count -gt 0 ]; do
    echo "Countdown: $count"
    count=$((count - 1))
    sleep 1
done
echo "Blast off!"

# Read input until a condition is met
while true; do
```

```
    read -p "Enter a command (type 'exit' to quit): " cmd

    if [ "$cmd" = "exit" ]; then
        break
    fi

    eval $cmd
done
```

Functions

Defining and Calling Functions

```
#!/bin/bash

# Define a simple function
say_hello() {
    echo "Hello, $1!"
}

# Function with return value
is_service_running() {
    service_name=$1

    if systemctl is-active --quiet $service_name; then
        return 0  # Success (service is running)
    else
        return 1  # Failure (service is not running)
    fi
}

# Call the function
say_hello "DevOps Engineer"

# Use function with conditional
if is_service_running "nginx"; then
    echo "Nginx is running"
else
    echo "Nginx is not running"
fi
```

Input/Output Redirection

Redirecting Output

```
#!/bin/bash

# Redirect standard output to a file
echo "Log entry at $(date)" > app.log

# Append output to a file
echo "Additional log entry" >> app.log

# Redirect errors to a file
ls /nonexistent 2> error.log

# Redirect both standard output and errors
ls /var/log/* > output.log 2> error.log

# Redirect both to the same file
ls -la > combined.log 2>&1
```

Reading Input from Files

```
#!/bin/bash

# Read a file line by line
while read line; do
    echo "Processing: $line"
done < servers.txt

# Use input as command arguments
cat commands.txt | xargs -I {} sh -c {}
```

Practical DevOps Scripts

System Monitoring Script

```
#!/bin/bash

log_file="system_status.log"
```

```
echo "===== System Status Report =====" > $log_file
echo "Date: $(date)" >> $log_file
echo "" >> $log_file

echo "CPU Usage:" >> $log_file
top -bn1 | grep "Cpu(s)" >> $log_file

echo "" >> $log_file
echo "Memory Usage:" >> $log_file
free -h >> $log_file

echo "" >> $log_file
echo "Disk Usage:" >> $log_file
df -h >> $log_file

echo "" >> $log_file
echo "Active Services:" >> $log_file
systemctl list-units --type=service --state=running | head -n 10 >>
$log_file

echo "Report generated at system_status.log"
```

Application Deployment Script

```
#!/bin/bash

APP_DIR="/var/www/myapp"
BACKUP_DIR="/var/backups/myapp"
REPO="https://github.com/username/myapp.git"

# Function to log messages
log_message() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1"
}

# Create backup of current application
create_backup() {
    log_message "Creating backup of current application..."
    timestamp=$(date +%Y%m%d_%H%M%S)
    mkdir -p $BACKUP_DIR
```

```
    if [ -d "$APP_DIR" ]; then
        tar -czf "$BACKUP_DIR/backup_$timestamp.tar.gz" -C "$APP_DIR" .
        log_message "Backup created at
$BACKUP_DIR/backup_$timestamp.tar.gz"
    else
        log_message "No existing application to backup"
    fi
}

# Deploy new version
deploy() {
    log_message "Deploying new version..."

    # Create app directory if it doesn't exist
    mkdir -p $APP_DIR

    # Clone repository
    git clone $REPO /tmp/myapp

    # Copy files to app directory
    cp -r /tmp/myapp/* $APP_DIR/

    # Clean up
    rm -rf /tmp/myapp

    log_message "Deployment completed"
}

# Restart services
restart_services() {
    log_message "Restarting services..."
    systemctl restart nginx
    systemctl restart php-fpm
    log_message "Services restarted"
}

# Main script execution
log_message "Starting deployment process"

create_backup
deploy
restart_services
```

```
log_message "Deployment process completed successfully"
```

Docker Container Health Check

```
#!/bin/bash

# List of containers to check
containers=("webapp" "database" "cache")

# Function to check if container is running
check_container() {
    container_name=$1

    # Check if container exists and is running
    if docker ps --format '{{.Names}}' | grep -q "^$container_name$"; then
        echo "✅ Container $container_name is running"
        return 0
    elif docker ps -a --format '{{.Names}}' | grep -q "^$container_name$";
then
        echo "❌ Container $container_name exists but is not running"
        return 1
    else
        echo "❌ Container $container_name does not exist"
        return 2
    fi
}

# Function to restart container
restart_container() {
    container_name=$1
    echo "🔄 Attempting to restart $container_name..."

    if docker start $container_name; then
        echo "✅ Successfully restarted $container_name"
        return 0
    else
        echo "❌ Failed to restart $container_name"
        return 1
    fi
}
```

```
# Main script
echo "==== Docker Container Health Check ===="
echo "Time: $(date)"
echo ""

for container in "${containers[@]}"; do
    if ! check_container $container; then
        restart_container $container
    fi
    echo ""
done

echo "==== Health Check Complete ===="
```

Debugging Techniques

Basic Debugging

```
#!/bin/bash

# Enable debug mode
set -x  # Print commands and their arguments as they are executed

# Your script commands here
echo "Debugging example"
name="DevOps"
echo "Hello, $name!"

# Disable debug mode
set +x

# Verbose mode
set -v  # Print shell input lines as they are read

# Your script commands here
echo "More commands"

# Disable verbose mode
set +v
```

Error Handling

```bash
#!/bin/bash

# Exit on error
set -e

# Function to clean up on exit
cleanup() {
    echo "Cleaning up temporary files..."
    rm -f /tmp/temp_file*.txt
}

# Register the cleanup function to be called on exit
trap cleanup EXIT

# Function to handle errors
handle_error() {
    echo "Error occurred at line $1"
    exit 1
}

# Register the error handler
trap 'handle_error $LINENO' ERR

# Create temporary files
echo "Creating temporary files..."
touch /tmp/temp_file1.txt
touch /tmp/temp_file2.txt

# This command will fail and trigger error handling
cat /nonexistent_file.txt

echo "This line will not be executed due to the error"
```

Understanding the Shebang in Bash Scripts

The shebang (also called hashbang) is the `#!` character sequence at the very beginning of a script file, followed by the path to the interpreter that should execute the script.

What is a Shebang?

bash

```
#!/bin/bash
```

In this example, `/bin/bash` is the path to the Bash interpreter on most Unix-like systems.

Why is the Shebang Useful?

1. **Interpreter Specification**: It explicitly tells the system which interpreter should run the script - whether it's Bash, Python, Perl, or another scripting language.
2. **Script Execution**: When you make a script executable (using `chmod +x script.sh`) and run it directly (`./script.sh`), the system uses the shebang to determine which program should interpret the file contents.
3. **Consistency**: It ensures the script runs with the intended interpreter regardless of the user's default shell.
4. **Portability**: It helps scripts work across different environments where the default shell might vary.
5. **Different Bash Versions**: You might need specific features from a particular Bash version, and the shebang ensures the correct version is used.

Is the Shebang Mandatory?

No, the shebang is not mandatory, but it's highly recommended for several reasons:

- **Without a shebang**, when you execute a script directly, it will run using your current shell's rules and features, which might not be what you intended.
- If you always execute the script by explicitly specifying the interpreter (`bash script.sh` instead of `./script.sh`), then the shebang isn't used.
- For non-executable scripts or those always run with an explicit interpreter command, the shebang is ignored.

Shebang Variations

1. **Bash-specific**:
   bash

```
#!/bin/bash
```

2. **Environment-independent** (uses the `env` command to locate bash):
   bash
   ```
   #!/usr/bin/env bash
   ```
   This is more portable across systems where bash might be in different locations.
3. **With options**:
   bash
   ```
   #!/bin/bash -e
   ```
   This runs bash with the `-e` flag (exit immediately if a command fails).

Best Practices

- Always include a shebang in scripts intended to be executed directly
- Use `#!/usr/bin/env bash` for better portability across different Unix-like systems
- Choose the appropriate interpreter based on your script's requirements
- Consider adding relevant options after the interpreter path if needed

In DevOps environments where scripts might run on various systems, using a proper shebang is especially important for consistent and reliable automation.

Best Practices for DevOps Bash Scripting

1. Use descriptive variable and function names
2. Add comments to explain complex logic
3. Handle errors and edge cases
4. Use consistent indentation and formatting
5. Create modular scripts with reusable functions
6. Always validate user input
7. Include proper logging
8. Use version control for your scripts
9. Add clear usage information
10. Test scripts in isolated environments before using in production

How Bash Reduces Work in DevOps Roles

1. **Automated Deployments**: Push code to production environments consistently with just one command
2. **Configuration Management**: Apply standardized configurations across multiple servers
3. **System Monitoring**: Automatically check system health and notify when problems arise

4. **Log Analysis**: Extract and analyze critical information from log files
5. **Backup and Recovery**: Schedule and verify regular backups without manual intervention
6. **Environment Provisioning**: Set up new environments with consistent configurations
7. **Continuous Integration**: Integrate with CI/CD pipelines to automate testing and deployment
8. **Task Scheduling**: Automate recurring administrative tasks
9. **Resource Scaling**: Respond to increased demand by provisioning additional resources
10. **Security Automation**: Regularly scan for vulnerabilities and apply patches

Conclusion

Bash scripting is a powerful tool in the DevOps arsenal. By mastering these fundamentals, you can automate repetitive tasks, ensure consistency across environments, and focus on more strategic aspects of your role. Start with simple scripts and gradually build more complex automation to significantly enhance your productivity and reliability as a DevOps engineer.